Resolving Common Modeling Issues Activity Book

Data Modeling for Qlik Sense

© 2023 QlikTech International AB. All rights reserved. All company and/or product names may be trade names, trademarks and/or registered trademarks of the respective owners with which they are associated.

DMQS May 2023

Table of Contents

Resolving Common Modeling Issues Activity Book

Table of Contents

Activity 1: Causing and resolving a synthetic key

Challenge 3
Hints 3
Part 1 - Causing a synthetic key 3
Progress check 4
Part 2 - Resolving a synthetic key 4
Solution 5

Activity 2 - Challenge: Load the Divisions and Categories tables

Challenge 6
Hints 6
Solution 6

Activity 3: Using Sum() vs Count()

Challenge 7
Hints 7
Solution 9

Activity 4: Active customers and number of orders

Challenge 10
Hints 10
Solution 11

Activity 5: Resolve a circular reference

Challenge 11
Hints 12
Solution 13

Activity 6: Load XML data

Challenge 13
Hints 14
Solution 15

Activity 7: Qualify and Unqualify

Challenge 15
Hints 16

Activity 1: Causing and resolving a synthetic key

Summary:

- Load another table of data from the ABC connection.
- Observe and analyze the resulting synthetic key.
- Resolve the synthetic key by aliasing a field.

Qlik Sense app: ABC Sales Data

Challenge

In the **ABC Sales Data** app, load the table *products* in the data model, by doing the following:

- Navigate to the **Products** script section of the **Data load editor**.
- Use the data connection to load all the fields from the table.
- In the script, name the new table *Products* and add a description.
- Load your data, and identify the synthetic key in your data model.
- In the **Data model viewer**, change data model views to see how Qlik Sense reads and stores data.
- In the **Data load editor**, resolve the synthetic key by aliasing *UnitPrice* as *CataloguePrice*.
- Reload the script and, using the **Data model viewer**, confirm there are no more data modeling issues.

Hints

- The **Select data** dialog in the data connection can be used to load the table.
- The **Source table view** can be used to see how Qlik Sense reads the tables.
- The **Internal table view** can be used to see how Qlik Sense stores data internally.
- The **Data load editor** should be used to alias fields.

Part 1 - Causing a synthetic key

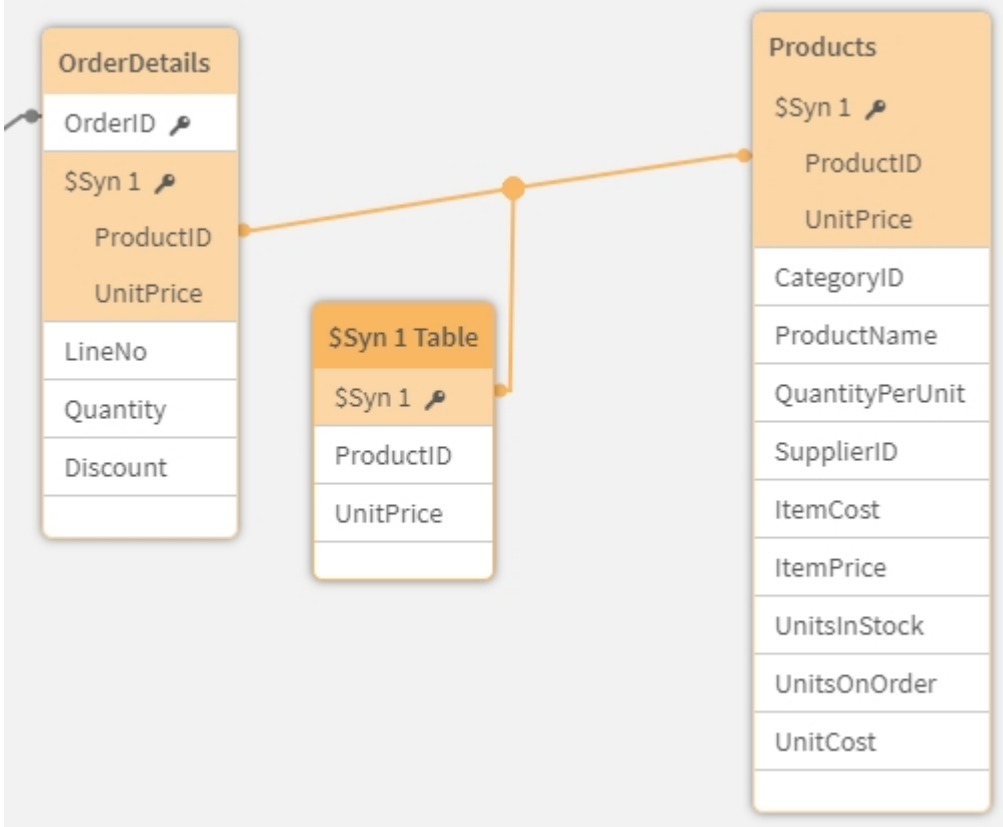
Step-by-step guide

1. In the **Data load editor**, navigate to the **Products** script section.
2. Use the **ABC** database connection to load the *products* table and make sure all the fields are selected.
3. Relocate the table label *[products]:* from above the **SELECT** statement, to above the preceding **LOAD** statement and capitalize the "P".
4. Add a comment to describe the table.
   Your script should appear as follows:

```
            // **** Load Products table ****
[Products]:
LOAD CategoryID,
...
```

5. Load your data and check the issues in the load script. Qlik Sense warns of a synthetic key being generated because both *ProductID* and *UnitPrice* appear in more than one table.
6. Open the **Data model viewer** and expand the tables. The data model shows the *$Syn 1 Table* that has been introduced.
7. Select the *$Syn 1* field to see the tables associated with the synthetic key.
8. Select one of the tables with a *$Syn 1* field. View the details of that table by expanding the **Preview** panel. Notice how the synthetic fields are not displayed in the preview of data.
9. The **Data model viewer** allows you to view how a data structure looks from the original data source. Click **Source table view** and note the difference. The source shows two links between the tables, which is not allowed in Qlik Sense.
10. Switch back to the **Internal table view** mode.

Progress check



Part 2 - Resolving a synthetic key

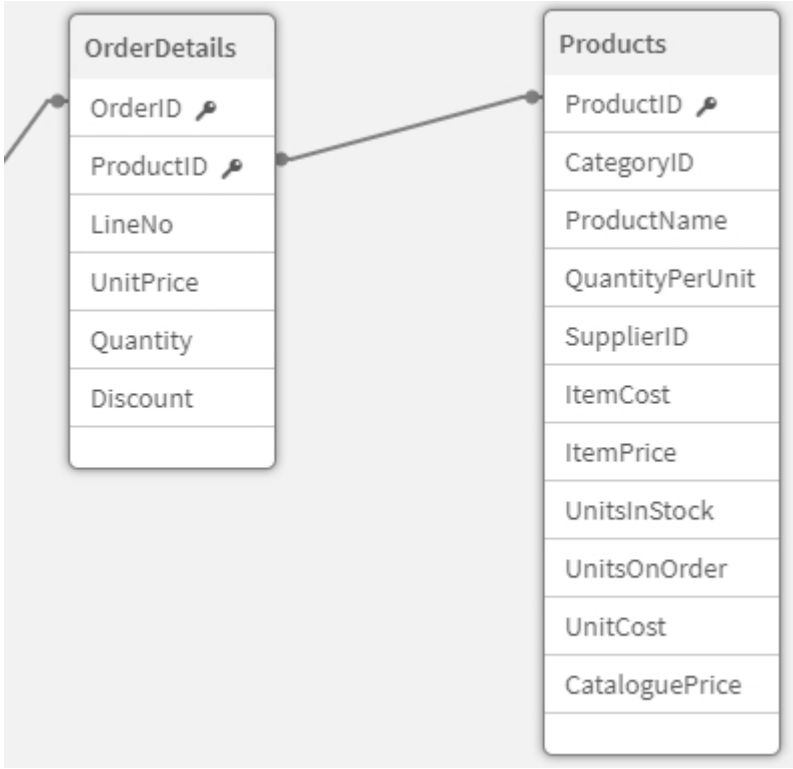
Step-by-step guide

1. In the **Data load editor**, select the **Products** script section.
2. Edit the line for the *UnitPrice* field in the preceding **LOAD** part of the *Products* table to alias the field as *CataloguePrice* using **AS**. Make sure you use the correct syntax at the end of the line. The script should appear as the following:

```
...
UnitCost,
UnitPrice AS CataloguePrice;
...
```

3. Load your data and in the **Data load progress** dialog, confirm that the synthetic key is no longer present in the data model.
4. Check the data model and verify that the synthetic key has been removed from the model and only the correct association, using *ProductID* between *OrderDetails* and *Products*, remains.

Solution



Activity 2 - Challenge: Load the Divisions and Categories tables

Summary:

- Load two dimension tables from the ABC connection.
- Verify the loaded tables are associated correctly to the existing model.

Qlik Sense app: ABC Sales Data

Challenge

In the **ABC Sales Data** app, add new tables to your data model by doing the following:

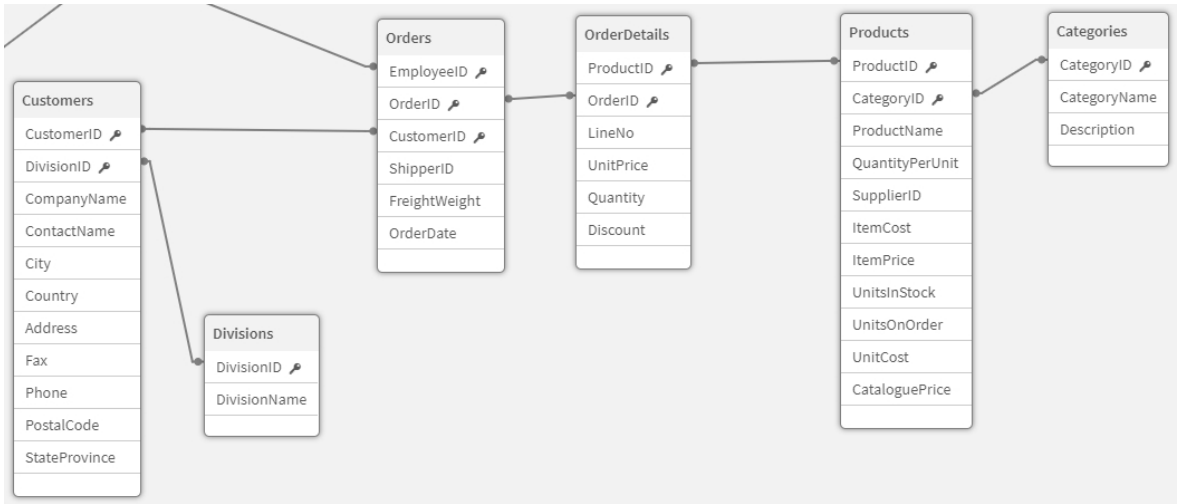
- Navigate to the **Products** script section of the **Data load editor**. Load the *categories* table from the **ABC_2025** database at the end of the script.
- Remove the extraneous **LIB CONNECT** between the *Product* and *Categories* tables.
- Navigate to the **Customers** script section of the **Data load editor**. Load the *Divisions* table from the **ABC_2025** database at the end of the script.
- Remove the extraneous **LIB CONNECT** between the *Customers* and *Divisions* tables.
- Label the tables appropriately, and add appropriate description as comments.
- Load the data and check the **Data model viewer** to ensure correct associations between the tables.

Note: The objective of this **challenge** is to test whether you feel comfortable with the concepts that have been discussed so far in your class. There is no **Step-by-Step** section in this activity, so you must only rely on the **Hints** below to understand the tasks you have to execute.
As you are still new to many of these concepts, please don't hesitate to reach out for more guidance (help file, instructor, Qlik Community).

Hints

- The new tables act as further dimensions in the data model.
- The **Select data** dialog in the data connection can be used to load the table.
- Load all the fields from the tables and ensure that preceding loads also exist.

Solution



Activity 3: Using Sum() vs Count()

Summary:

- Analyze current record numbers for Orders and OrderDetails.
- Create visualizations to confirm data model records.
- Modify script for the Orders table to provide counter fields.

Qlik Sense app: ABC Sales Data

Challenge

Count the number of orders in your current data model. Be prepared to explain why using **Count** is potentially risky and why using **Sum** is more efficient by doing the following:

- Verify the number of orders using the **Preview** panel in the **Data model viewer**.
- In the **App overview**, create a new sheet named *Basic data transformations*.
- In sheet edit mode, toggle **Advanced options** on.
- Create a **Text & image** object that contains the text *Count(OrderID)*, and the same as an expression.
- In the same sheet, create a second **Text & image** object that contains the text *Count(Distinct OrderID)*, and the same as an expression.
- In the **Orders** script section, create a new field in the *Orders* table: `OrderID AS OrderRecordCounter,` and reload your data.
- In the sheet, copy and paste the first object you created for a third **Text & image** object that contains the text *Count(OrderRecordCounter)*, and the same as an expression.
- In your script, modify the new field in the *Orders* table as follows: `1 AS OrderRecordCounter,` and reload your data.
- Modify the text and the expression in the last **Text & image** object to: *Sum(OrderRecordCounter)*, and observe the results.

Hints

- Counting key fields in your data model using the **Count()** function should be avoided.
- Using **Sum()** instead of **Count()** is recommended for counting records.

Step-by-step guide

1. Open your **Data model viewer** and use the **Preview** panel to check the number of rows in the *Orders* table. This is the number of orders you have in your data set. Verify that the number is *6571* rows (orders).
2. Use the **Analyze Sheet** option to edit a blank sheet and change the title to *Basic data transformations*.

Use Count(OrderID)

3. With **Advanced options** toggled on, remove any existing objects from the new sheet, then open the **Charts** in the **Assets** panel. Drag a **Text & image** object onto the sheet and resize it to be **16 columns x 1 row**. Place it at the top left corner of your sheet.
4. Enter the text *Count(OrderID):* in the **Text & image** object.
5. From the object **Properties** window on the right, add a new measure. Use the *OrderID* field and select *Count(OrderID)* from the aggregation options. The result of this calculation is *20246*. It is different than the result you verified in Step 1.

> **Note: Count()** is performed over all the possible values of the *OrderID* field from all the tables that contain it. So in this case, the **Count()** function counts the records in the *Order-Details* table, which is the longest of the two tables that contain the field.

Use Count(Distinct OrderID)

6. Select the **Text & image** object you created and make a copy of it on the same sheet.
7. Resize the new object to match the previous one. Place it under the first object.
8. Edit the text of the object to modify it as follows: *Count(Distinct OrderID):*
9. From the **Properties** window, expand the measure properties for **Count(OrderID)** and modify the expression to: *Count(Distinct OrderID)*.
10. Disable the edit mode in your sheet and check the results. The new expression that contains the *Distinct* qualifier shows *6571*, which is the correct and expected number of rows.

> **Note:** The **Count(Distinct...)** syntax will give you the correct result. However, according to best practices, **Count(Distinct...)** is a heavy function and, therefore, many occurrences should be avoided in the front-end of your app, as it will affect the performance.

Use OrderID AS OrderRecordCounter

11. In the **Data load editor**, navigate to the **Orders** script section and add the following line in the *Orders* table, under the existing *OrderID* field:

```
OrderID AS OrderRecordCounter,
```

This will create a counter field to count the number of orders.

> **Note:** This field is related to orders at an order level, so it is added to the *Orders* table. It is not at an order-line level, so you do not add it to *OrderDetails*.

12. Load your data to add the new field *OrderRecordCounter* into the data model.
13. In the **Data model viewer**, verify that the new field, *OrderRecordCounter*, exists in the data model within the *Orders* table. Preview the table to verify that the *OrderRecordCounter* field contains the exact same values as the *OrderID* field.
14. In **Analyze** mode, open the **Basic data transformations** sheet and enable the edit mode, if required.
15. Copy the first **Text & image** object you added into the field. Resize it to be exactly the same and place it under the second object.
16. Edit the text in the object and modify it as follows: *Count(OrderRecordCounter):*

17. From the **Properties** window, expand the measure properties for **Count(OrderID)** and modify the expression to: *Count(OrderRecordCounter)*.
18. Disable the edit mode in your sheet and check the results. The new expression is *6571* which, again, is the correct and expected number of rows.

> **Note:** Despite the fact that the **Count(...)** function is more efficient than the **Count (Distinct ...)** function (simpler algorithm), whenever possible, you should avoid using **Count()** on the front-end. The **Sum()** function, which is much faster, can be used instead.

Use 1 AS OrderRecordCounter

19. In the **Data load editor**, navigate to the **Orders** section. Comment out the existing counter field in the *Orders* table and create a new counter with *1* instead of *OrderID*, as seen below:

```
//  OrderID AS OrderRecordCounter,
    1 AS OrderRecordCounter,
```

20. Load your data to update the values of the new field, *OrderRecordCounter*, into the data model.
21. In the **Data model viewer**, verify that the field, *OrderRecordCounter*, exists and now contains the value 1.
22. From the **App overview** window, open the **Basic data transformations** sheet and enable the edit mode again, if required.
23. Edit the third **Text & image** object you added into the field, and modify its text as follows: *Sum (OrderRecordCounter):*
24. From the **Properties** window, expand the measure properties for **Count(Order-RecordCounter)** and modify the expression to: *Sum(OrderRecordCounter)*.
25. Disable the edit mode in your sheet and check the results. The result of the new expression is *6571* which, again, is the correct and expected number of rows.

> **Note:** According to best practices, using **Sum()** is the recommended way to add counter fields into your tables.

Solution

Activity 4: Active customers and number of orders

Summary:

- Add a counter field to the Orders table to track active customers.
- Create a new table visualization to track number of active customers and orders, by division.

Qlik Sense app: ABC Sales

Challenge

Create a new visualization in the **ABC Sales Data** app to show how many active customers there are and the number of orders, by division. To achieve this, do the following:

- Create a new field in the *Orders* table named *ActiveCustomerID*.
- Load your data to add it into the data model. The new field contains the *CustomerID* values.
- In the **Basic data transformations** app sheet, insert a **Table** and add three columns:
  - *DivisionName* as dimension.
  - *Count(Distinct ActiveCustomerID)* as measure, with *Active Customers* as the label.
  - *Sum(OrderRecordCounter)* as measure, with *Orders* as the label.

Hints

- The field *ActiveCustomerID* refers to the active customers who have placed an order. This is why it is added to the *Orders* table and not the *Customers* table.

Step-by-step guide

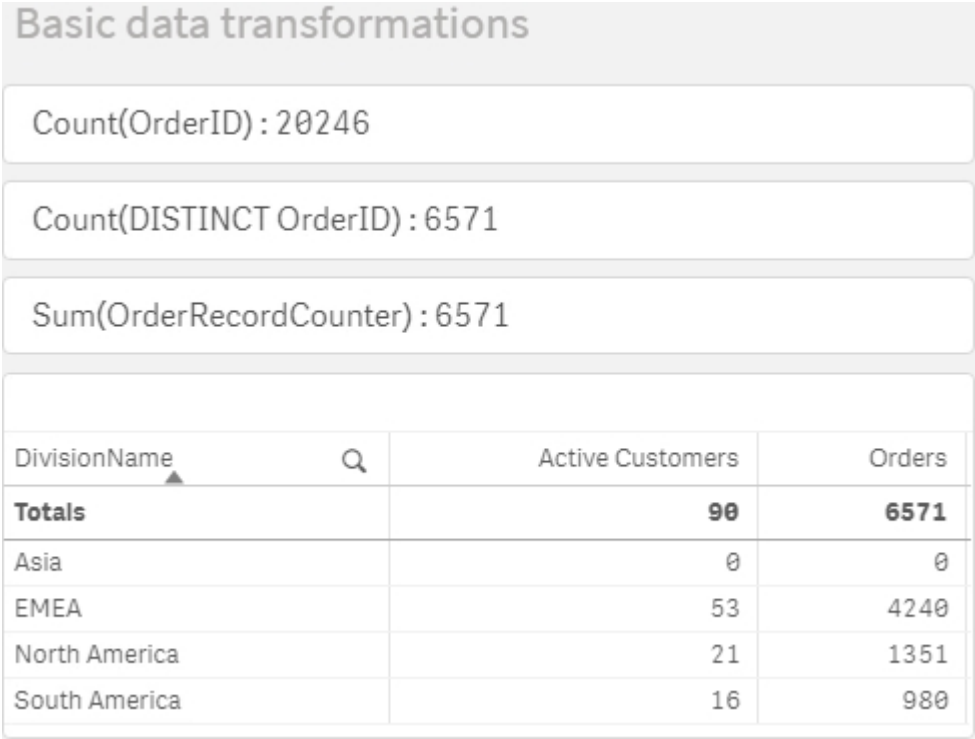
1. In the **Data load editor**, navigate to the **Orders** section and create a new counter field in the *Orders* table for the number of active customers:

```
            // **** Load Orders table ****
[Orders]:
LOAD OrderID,
    //  OrderID AS OrderRecordCounter,
    1 AS OrderRecordCounter,
    CustomerID,
    CustomerID AS ActiveCustomerID,
    ...
```

2. Load your data to add the new field into the data model.
3. Navigate to the **Analyze** mode and open the **Basic data transformations** sheet, if applicable.
4. Enable the edit mode on the sheet, if required.
5. From the **Charts** tab in the **Assets** panel, drag a **Table** object onto the sheet, aligned with the **Text & image** objects.
6. Use the **Fields** tab in the **Assets** panel to add the *DivisionName* field to the table as a dimension.
7. Locate and add the *ActiveCustomerID* field to the table as a measure. From the aggregations list, select the *Count(ActiveCustomerID)*.

8. From the **Properties** panel, modify the new expression to *Count(Distinct ActiveCustomerID)*, and change the label to *Active Customers*.
9. In the **Assets** panel, locate and add the *OrderRecordCounter* field to the table as another measure. In the aggregation list, select *Sum(OrderRecordCounter)*. In the **Properties** panel, change the label to *Orders*.
10. Disable the edit mode to finish editing and verify that the table you just created shows the breakdown of how many *Active Customers* and *Orders* there are for each *DivisionName*.

Solution



DivisionName	Active Customers	Orders
Totals	**90**	**6571**
Asia	0	0
EMEA	53	4240
North America	21	1351
South America	16	980

Activity 5: Resolve a circular reference

Summary:

- Load a table of shippers into the data model.
- Examine the structure of the model and observe the circular reference generated.
- Resolve the issue by aliasing a field.

Qlik Sense app: ABC Sales Data

Challenge

In the **ABC Sales Data** app, load a table and eliminate the circular reference in your data model by doing the following:

- Create a new *Shipments* script section and load the *shippers* table from the **ABC_2025** database.
- Label the table *Shippers* and add a comment to describe its contents.
- Load your data and notice the circular reference in your data model.
- In the *Shippers* table, rename the *CompanyName* field to *ShipperName* to eliminate the circular reference.
- Load your data and verify that the circular reference has been eliminated from your data model.

Hints

- The loops (circular references) must always be eliminated from the data model because they will cause data integrity issues.
- Because of the circular reference in the data model, the *OrderDetails* has been converted into a loosely coupled table as it is the longest table.

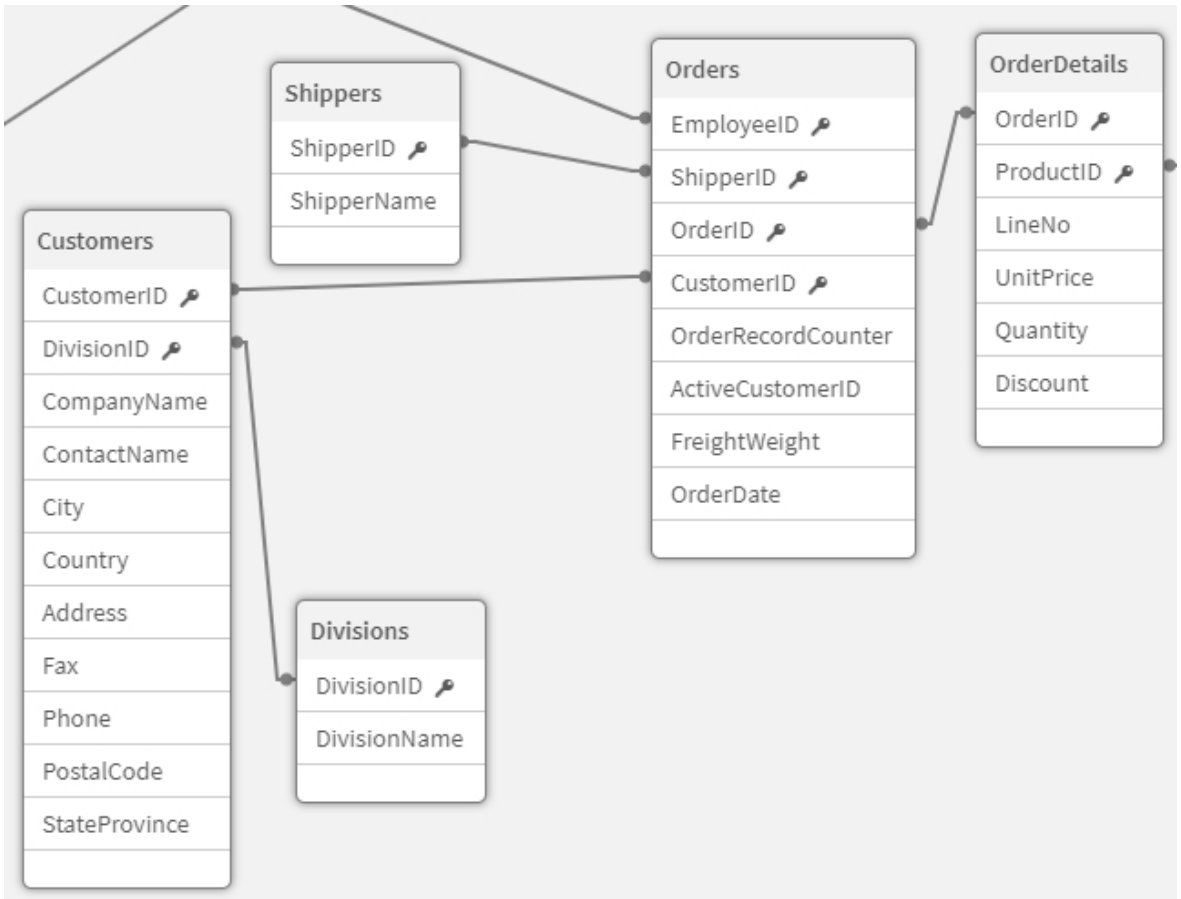
Step-by-step guide

1. Create a new script section after **Employees**, and title it *Shipments*.
2. Use the **ABC** database connection to load the *shippers* table and make sure all the fields are selected.
3. Relocate the table label **[shippers]** to above the preceding **LOAD** statement and capitalize the "S".
4. Add a comment to describe its contents.
5. Load the data and notice the error for the circular reference.
6. Examine your data structure in the **Data model viewer**. A circular reference is shown because both the *Customers* and *Shippers* tables have a *CompanyName* field. These are not related. So, one can be aliased as another name.
7. In the *Shippers* table loading script, alias the field as follows:

   ```
   CompanyName AS ShipperName;
   ```

8. Load your data again and verify that there is no circular reference after the transformation.

Solution



Activity 6: Load XML data

Summary:

- Load data for suppliers from an .xml file.
- Observe the synthetic table and circular reference caused by the field naming convention clash.

Qlik Sense app: ABC Sales Data

Challenge

In the **ABC Sales Data** app, load a new table from an .xml file by doing the following:

- Create a new script section, *Suppliers*, between the **Employees** and **Shipments** sections.
- Use the **DataFiles** connection to import *suppliers.xml* into your cloud tenant.

Qlik Sense on Windows: Create a new data connection, named *XMLFiles*, to the **XMLFiles** folder, under **C:\Qlik Sense Data**.

- Load the following tables in the **Suppliers** section:
  - *root/suppliers* table from the *suppliers.xml* file and label the table *Suppliers*.
  - *root/supplier_band* table in the same file and label the table *SupplierBand*.
  - Add comments for both tables.
- Load your data and notice that a synthetic key and a circular reference appear in your data model. Explain why.

Hints

- *Suppliers* and *Customers* tables have multiple fields in common.
- When the **Select data** dialog appears, the .xml file format must be selected.

Step-by-step guide

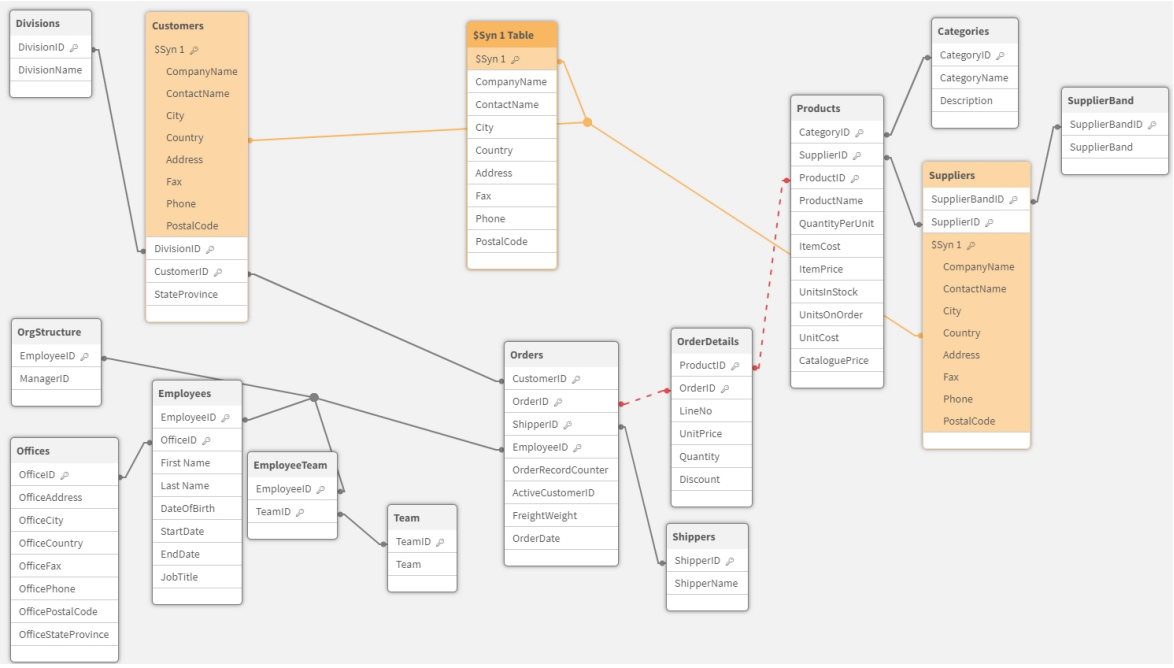
1. In the **Data load editor**, create a new section and name it *Suppliers*. Place **Suppliers** between the **Employees** section and the **Shipments** section.
2. From the **Data connections** panel, use **Select data** under the **DataFiles** connection.
3. Sourcing the data from the student file collateral, drag the *suppliers.xml* file onto the drop zone to import it.

   Qlik Sense on Windows: Create a new data folder connection, named *XMLFiles*, to the **XMLFiles** folder, under **C:\Qlik Sense Data**.
4. Use the data connection to select the *suppliers.xml* as your data file.
5. In the **Select data** dialog, notice that there are only two tables:
   - *root/suppliers*
   - *root/supplier_band*
6. Ensure that all the fields from both tables are checked and insert the script.
7. Label the new tables *Suppliers* and *SupplierBand*, respectively, and add a suitable comment above each table.
8. Load your data and check your data model in the **Data model viewer**. Notice that a synthetic key and a circular reference have been created.
9. Identify the fields that cause the synthetic key by selecting the *$Syn 1 Table*.

   The synthetic key and circular reference are created because the file *suppliers.xml* has several fields in common with the *Customers* table loaded from the database. These fields must not be associated with each other. Some of the field names in the *Suppliers* table must be changed.

   Instead of aliasing each field using the **AS** keyword or other renaming statements, the simplest solution is to have the table name precede the field name (which is commonly seen in most database utilities). You can easily accomplish this by using a special statement in Qlik Sense that qualifies all the field names with the name of the table when they are loaded. This strategy is used in Activity 7 in this Module.

Solution



Activity 7: Qualify and Unqualify

Summary:

- Use the Qualify and Unqualify script statements to automatically qualify the supplier field names.
- Reload the script and observe the fixes within the data model.

Qlik Sense app: ABC Sales Data

Challenge

Use **QUALIFY** and **UNQUALIFY** to resolve the synthetic keys and circular reference in your data model.

- In the **Suppliers** script section, before the *Suppliers* table, qualify all the fields in the table.
- Unqualify the *SupplierID* and *SupplierBandID* to maintain the correct associations.
- After the table **LOAD** statement, stop qualifying all the fields.
- Load your data and verify that the table *Suppliers* exists and all the issues have been eliminated.

Hints

- Unqualify *SupplierID* and *SupplierBandID* so they link with the data model.
- If the **UNQUALIFY** statement is not used after the **LOAD** statement, the script will continue to qualify all the fields that are loaded after the **QUALIFY** and this would lead to breaking all the associations.

Step-by-step guide

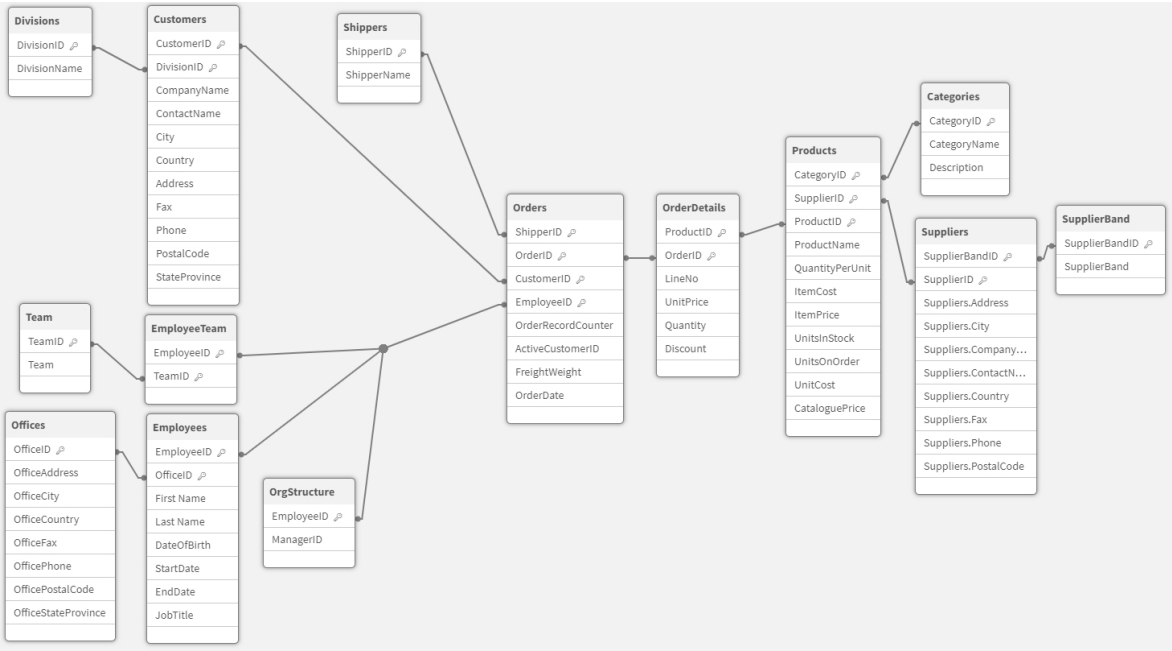
1. In the **Data load editor**, navigate to the **Suppliers** section.
2. Qualify all the fields in the *Suppliers* table, except for *SupplierID* and *SupplierBandID*, by adding the following two lines before the *Suppliers* table **LOAD** statement and table label.

```
QUALIFY *;

UNQUALIFY SupplierID, SupplierBandID;
```

3. Switch off qualification by adding the following line after the *Supplier* table **LOAD** statement, but before the *SupplierBand* table **LOAD** statement:

```
UNQUALIFY *;
```

4. Load your data and check your data model in the **Data model viewer**. Verify that there are no synthetic keys or circular references.

Solution